

Dokumentacja Wstępna

Algorytmy zaawansowane - projekt 2025/2026 Problem komiwojażera krótkodystansowca (Bottleneck TSP)

Piotr Tyrakowski, Sandra Adamiec, Hubert Sobociński

April 14, 2026

1 Przedstawienie problemu

Zadanie polega na stworzeniu aproksymacyjnego algorytmu dla problemu komiwojażera krótkodystansowca (ang. *Bottleneck TSP*), w którym celem jest minimalizacja **najcięższej krawędzi** na trasie. Zakładamy, że wejściowy graf pełny spełnia nierówność trójkąta. Oczekiwany algorytm to 3-aproksymacja, co oznacza, że znaleziona najdłuższa krawędź cyklu będzie co najwyżej trzykrotnie dłuższa od krawędzi optymalnej.

2 Opis rozwiązania problemu

Algorytm dzieli się na dwa główne etapy:

Etap 1: Wyznaczenie Minimalnego Drzewa Rozpinającego (MST)

Minimalne drzewo rozpinające budowane jest za pomocą algorytmu Prima operującego bezpośrednio na macierzy wag. Algorytm utrzymuje tablicę najniższych kosztów dołączenia każdego wierzchołka do rosnącego drzewa i w każdej iteracji wybiera najtańszy wierzchołek liniowym przeglądem tej tablicy. Cały etap wykonuje się w czasie $O(V^2)$.

Etap 2: Konstrukcja cyklu (własność sześcianu drzewa T^3)

Zamiast fizycznie budować sześcian grafu, algorytm używa metody "dziel i zwyciężaj", operując bezpośrednio na wygenerowanym drzewie MST:

1. **Podział:** Usuwamy dowolną krawędź (u, v) , dzieląc drzewo na dwa poddrzewa.
2. **Rekurencja:** W obu poddrzewach wywołujemy szukanie ścieżek zaczynających się w punktach cięcia $(u \text{ i } v)$, a kończących w ich dowolnych sąsiadach (u_1, v_1) . Baza rekurencji to trywialne ułożenie ścieżki dla poddrzew o rozmiarze ≤ 3 wierzchołków.
3. **Sklejanie:** Zwrócone ścieżki są sklejane w jedną. Z matematycznej własności T^3 wynika, że wirtualny skok łączący u_1 z v_1 wymaga pokonania maksymalnie 3 krawędzi MST.
4. **Zamknięcie cyklu:** Na najwyższym poziomie rekurencji łączymy początek i koniec uzyskanej ciągłej ścieżki pierwotnie usuniętą krawędzią (u, v) , zamykając cykl Hamiltona.

3 Analiza poprawności i jakości (Dowód)

Gwarancję poprawności algorytmu zapewnia twierdzenie o sześciu grafu spójnego.

Twierdzenie 1. *Sześciu dowolnego spójnego drzewa T (oznaczany jako T^3) posiada cykl Hamiltona.*

Dowód (indukcja matematyczna ze względu na liczbę wierzchołków n). Baza indukcji: Dla $n \in \{1, 2, 3\}$, największa odległość między dowolnymi dwoma wierzchołkami w drzewie wynosi co najwyżej 2. Zatem sześciu takiego drzewa (T^3) staje się grafem pełnym (kliką), w którym znalezienie cyklu Hamiltona jest trywialne.

Założenie indukcyjne: Zakładamy, że dla każdego spójnego drzewa o liczbie wierzchołków $k < n$, w jego sześciu można wyznaczyć ścieżkę Hamiltona.

Krok indukcyjny: Rozpatrzmy dowolne drzewo T o n wierzchołkach.

1. Wybieramy z drzewa krawędź (u, v) i ją usuwamy, co dzieli drzewo na dwa rozłączne poddrzewa: T_u oraz T_v . Oba mają mniej niż n wierzchołków.
2. Na mocy założenia w T_u^3 odnajdujemy ścieżkę Hamiltona, która zaczyna się w u , a kończy w jego bezpośrednim sąsiedzie u_1 .
3. Analogicznie dla T_v^3 odnajdujemy ścieżkę, zaczynającą się w sąsiedzie v_1 , a kończącą w v .
4. Łączymy obie ścieżki skokiem z u_1 do v_1 . W oryginalnym drzewie T najkrótsza droga między nimi to: $u_1 \rightarrow u \rightarrow v \rightarrow v_1$ (dokładnie 3 krawędzie). Zatem krawędź (u_1, v_1) na pewno istnieje w grafie T^3 .
5. Cykl zamykamy krawędzią (u, v) . Na mocy indukcji konstrukcja jest zawsze poprawna.

□

Dowód współczynnika aproksymacji (3-aproksymacja):

1. Niech W_{opt} będzie wagą najcięższej krawędzi w **optymalnym** cyklu Hamiltona.
2. Jeśli z optymalnego cyklu usuniemy jedną krawędź, otrzymamy ścieżkę Hamiltona (drzewo rozpinające). Z definicji algorytmu Kruskala, najcięższa krawędź w zbudowanym przez nas MST (oznaczymy ją jako W) nie może być cięższa niż najcięższa krawędź w jakimkolwiek innym drzewie rozpinającym. Stąd $W \leq W_{opt}$.
3. Z przedstawionego wyżej dowodu wynika, że tworząc nowy cykl, łączymy wierzchołki pokonując wirtualnie maksymalnie **3 krawędzie** oryginalnego MST.
4. Ponieważ graf wejściowy spełnia **nierówność trójkąta**, waga bezpośredniego połączenia między dwoma wierzchołkami jest nie większa niż suma wag krawędzi na ścieżce łączącej je w MST.
5. Ścieżka ta składa się z co najwyżej 3 krawędzi, a każda z nich waży maksymalnie W . Zatem nowa krawędź w cyklu waży co najwyżej $3W$.
6. Zestawiając to z punktem 2, najcięższa krawędź w wygenerowanym przez nas cyklu ma wagę $\leq 3W \leq 3W_{opt}$. Algorytm zawsze znajduje rozwiązanie co najwyżej 3-krotnie gorsze od optymalnego.

4 Pseudokod

Poniższy pseudokod prezentuje główną logikę opisanego algorytmu.

Algorytm 1 Algorytm rozwiązujący Bottleneck TSP

```
1: Funkcja ROZWIĄŻBOTTLENECKTSP(Macierz  $W[V][V]$ )
2:    $Drzewo\_MST \leftarrow \text{ALGORYTMPRIMA}(W)$ 
3:   Niech  $(v, u)$  będzie dowolną krawędzią w  $Drzewo\_MST$ 
4:    $cieka \leftarrow \text{ZNAJDŹŚCIEŻKĘREKURENCYJNIE}(Drzewo\_MST, v, u)$ 
5:    $Cykl\_Wynikowy \leftarrow cieka + [v]$   $\triangleright$  Zamknięcie cyklu krawędzią  $(u, v)$ 
6:   Zwróć  $Cykl\_Wynikowy$ 
7: end Funkcja
```

Algorytm 2 Budowa MST algorytmem Prima z macierzą wag

```
1: Funkcja ALGORYTMPRIMA(Macierz  $W[V][V]$ )
2:    $w\_drzewie[0 \dots V-1] \leftarrow \{\text{false}\}$ 
3:    $min\_koszt[0 \dots V-1] \leftarrow \{\infty\}$ 
4:    $rodzic[0 \dots V-1] \leftarrow \{-1\}$ 
5:    $min\_koszt[0] \leftarrow 0$ 
6:   for  $i \leftarrow 0$  do  $V-1$  do
7:      $u \leftarrow$  wierzchołek spoza drzewa o najmniejszym  $min\_koszt$   $\triangleright$  Skan liniowy  $O(V)$ 
8:      $w\_drzewie[u] \leftarrow \text{true}$ 
9:     for  $v \leftarrow 0$  do  $V-1$  do
10:      Jeśli  $\neg w\_drzewie[v]$  i  $W[u][v] < min\_koszt[v]$  to
11:         $min\_koszt[v] \leftarrow W[u][v]$ 
12:         $rodzic[v] \leftarrow u$ 
13:      end Jeśli
14:    end for
15:  end for
16:  Zwróć drzewo wyznaczone przez tablicę  $rodzic$ 
17: end Funkcja
```

Algorytm 3 Konstrukcja ścieżki na podstawie MST i własności sześciangu

```
1: Funkcja ZNAJDŹŚCIEŻKĘREKURENCYJNIE(Drzewo, start, koniec)
2:   Jeśli  $\text{LiczbaWierzchołków}(Drzewo) \leq 3$  to
3:     Zwróć  $uloz\_wierzchołki\_w\_ścieżkę\_od(start, do: koniec)$ 
4:   end Jeśli
5:   Usuń krawędź  $(start, koniec)$  z  $Drzewo$ 
6:    $Poddrzewo\_start \leftarrow$  wyodrębnij_część_zawierającą( $Drzewo, start$ )
7:    $Poddrzewo\_koniec \leftarrow$  wyodrębnij_część_zawierającą( $Drzewo, koniec$ )
8:    $ssiad\_start \leftarrow$  pobierz_dowolnego_sasiada( $Poddrzewo\_start, start$ )
9:    $ssiad\_koniec \leftarrow$  pobierz_dowolnego_sasiada( $Poddrzewo\_koniec, koniec$ )
10:   $cieka\_L \leftarrow \text{ZNAJDŹŚCIEŻKĘREKURENCYJNIE}(Poddrzewo\_start, start, ssiad\_start)$ 
11:   $cieka\_P \leftarrow \text{ZNAJDŹŚCIEŻKĘREKURENCYJNIE}(Poddrzewo\_koniec, ssiad\_koniec, koniec)$ 
12:  Zwróć  $PołączListy(cieka\_L, cieka\_P)$   $\triangleright$  Skok o max 3 krawędzie w drzewie
13: end Funkcja
```

5 Złożoność czasowa

Złożoność obliczeniowa całego algorytmu dla grafu pełnego o V wierzchołkach wynosi $O(V^2)$, co wynika z następujących składowych:

- **Algorytm Prima (MST):** Operuje bezpośrednio na macierzy wag. W każdej z V iteracji wykonuje liniowy przegląd tablicy kosztów ($O(V)$) oraz aktualizację sąsiadów ($O(V)$). Łącznie: $O(V^2)$. Nie wymaga sortowania krawędzi ani struktury Union-Find.
- **Konstrukcja cyklu T^3 :** Algorytm rekurencyjny odwiedza każdy wierzchołek i każdą krawędź drzewa stałą liczbę razy. Złożoność tego etapu wynosi $O(V)$, pod warunkiem że ścieżki częściowe przechowywane są jako listy wiązane, co gwarantuje łączenie dwóch ścieżek (operacja `PołączListy`) w czasie $O(1)$. Użycie tablic dynamicznych spowodowałoby koszt kopiowania przy każdym sklejeniu, co w pesymistycznym przypadku (drzewo będące ścieżką) podniosłoby złożoność tego etapu do $O(V^2)$.

Dominującym składnikiem jest budowa MST, stąd złożoność całkowita: $O(V^2)$. Jest to zarazem optymalne dla grafu pełnego zapisanego macierzowo, ponieważ samo odczytanie danych wejściowych wymaga $\Theta(V^2)$ czasu.

6 Format pliku wejściowego

Dane wejściowe reprezentują pełny graf ważony w postaci macierzy wag.

```
n
w11 w12 ... w1n
w21 w22 ... w2n
...
wn1 wn2 ... wnn
```

gdzie:

- n – liczba wierzchołków,
- w_{ij} – waga krawędzi między wierzchołkami i i j ,
- $w_{ii} = 0$,
- macierz jest symetryczna,
- wagi spełniają nierówność trójkąta.

Przykład.

```
4
0 3 5 2
3 0 4 6
5 4 0 1
2 6 1 0
```